

Lua 程式設計

第五講

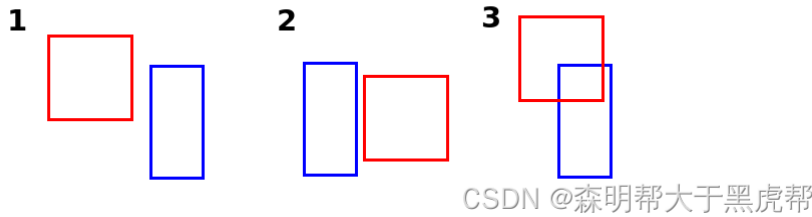


Love2D碰撞偵測

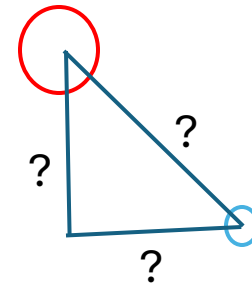
- 方式一:手搓
 - 方式二:使用內建物理引擎
-

Love2D碰撞偵測-手搓

- AABB 矩形碰撞偵測 —— 適合純 2D 塊狀 / 方塊遊戲



- function checkCollision(x1, y1, w1, h1, x2, y2, w2, h2)
- return $x1 < x2 + w2$ and $x2 < x1 + w1$ and $y1 < y2 + h2$ and $y2 < y1 + h1$
- end
- 圓形距離碰撞偵測 —— 適合球形、子彈或點狀判定
- function checkCircleCollision(x1, y1, r1, x2, y2, r2)
- local dx, dy = $x2 - x1$, $y2 - y1$
- local distance = $\text{math.sqrt}(dx * dx + dy * dy)$
- return distance < $(r1 + r2)$ end



Love2D碰撞偵測-手搓

- function canMove(shape, posX, posY)--建立canMove功能用於確認方塊是否可以移動
- for index, block in ipairs(shape) do
- local x = posX + block.x
- local y = posY + block.y
- if x < 1 or x > boardWidth then
- return false end
- if y > boardHeight then
- return false end
- if board[y] and board[y][x] == 1 then
- return false end
- end
- return true end

Love2D碰撞偵測-內建物理引擎Box2D

- love.physics 模組就是基於強大的 Box2D 物理引擎
- 思考如何模擬:球落到地面並彈起
- Box2D 的核心主要概念：
 - 世界 (World):物理宇宙。所有物體都必須註冊在同一個世界中，它們才會互相碰撞
 - 剛體 (Body,你是誰、你在哪):物件的位置、速度與旋轉度,是動態 (dynamic) 還是靜態 (static)。Body 本身在物理世界中是「隱形且沒有體積」的點
 - 形狀 (Shape,長什麼樣子):負責定義幾何大小（如圓形半徑、矩形寬高），它本身沒有物理特性
 - 夾具(Fixture,摸起來是什麼材質):是將 Shape 黏到 Body 上的「膠水」。透過 Fixture，我們才能設定物體的摩擦力、彈力 (Restitution) 和密度 (Density)

想像世界

```
1.  local world  local ball = {}  local ground = {} -- world:世界 ball:球 ground:地表
2.  function love.load()
3.      -- 1. 設定遊戲視窗大小
4.      love.window.setMode(800, 600)
5.      -- 2. 建立 Box2D 世界 (參數為：X軸重力,Y軸重力)
6.      world = love.physics.newWorld(0, 9.81 * 60, true)-- Y軸設為 9.81 * 60 (向下重力) · 讓物體有明顯的下墜感
7.      -- 3. 建立靜態地面 (Static Body)
8.      ground.body = love.physics.newBody(world, 400, 550, "static")    -- 位置在 (400, 550)
9.      ground.shape = love.physics.newRectangleShape(700, 20)          -- 寬 700, 高 20 的矩形
10.     ground.fixture = love.physics.newFixture(ground.body, ground.shape) -- 綁定身體與形狀
11.     -- 4. 建立動態小球 (Dynamic Body)
12.     ball.body = love.physics.newBody(world, 400, 100, "dynamic") -- 位置在 (400, 100)
13.     ball.shape = love.physics.newCircleShape(25)                    -- 半徑 25 的圓形
14.     -- 5.建立 Fixture 並設定密度 (Density)
15.     ball.fixture = love.physics.newFixture(ball.body, ball.shape, 1)
16.     ball.fixture:setRestitution(0.75) -- 設定彈力 (0 = 不彈, 1 = 完全彈回)
17. end
```

產生世界萬物

1. `function love.draw()`
2. `-- 繪製地面 (灰色矩形)`
3. `love.graphics.setColor(0.5, 0.5, 0.5)`
4. `-- 使用 getPoints 獲取矩形四個頂點的實際世界座標`
5. `love.graphics.polygon("fill",
 ground.body:getWorldPoints(ground.shape:getPoints()))`
6. `-- 繪製小球 (紅色圓形)`
7. `love.graphics.setColor(1, 0, 0)`
8. `-- 獲取小球目前的 X, Y 座標與半徑`
9. `love.graphics.circle("fill", ball.body:getX(), ball.body:getY(), ball.shape:getRadius())`
10. `end`

吹一口氣讓世界活起來

- function love.update(dt)
- -- 更新物理世界 (讓時間前進)
- world:update(dt)
- end

推一下球

1. `function love.keypressed(key, scancode)`
2. `local speed = 700` -- 單位是牛頓 (N) , 可以根據需要調整這個值來改變小球的移動速度
3. `if key == "left" then` -- 按下左鍵,代表給球一個往左邊的推力
4. `ball.body:applyForce(-speed, 0)`
5. `elseif key == "right" then`--按下右鍵,代表給球一個往右邊的推力
6. `ball.body:applyForce(speed, 0)`
7. `end`

拍一下球

```
1.  function love.keypressed(key, scancode)
2.      if scancode == "down" or scancode == "s" then --按下向下鍵,代表拍球
3.          -- 獲取小球目前的 X 與 Y 軸速度
4.          local vx, vy = ball.body:getLinearVelocity() --
5.          -- 設定拍球的衝量強度 ( 數值越大, 拍得越猛烈 )
6.          local slapImpulse = 1500
7.          -- 如果小球原本正在往上彈 (  $vy < 0$  ), 我們希望先抵消它的上升速度
8.          if vy < 0 then
9.              -- 強制將 Y 軸速度歸零, 讓拍擊感更扎實, 不會被上升的慣性抵消
10.             ball.graphics = ball.body:setLinearVelocity(vx, 0)
11.         end
12.         -- 施加向下的瞬間衝量 (X軸為 0, Y軸為正數值)
13.         -- 後面的參數是施力點, 設定在小球的質心
14.         ball.body:applyLinearImpulse(0, slapImpulse, ball.body:getWorldCenter())
15.     end
16. end
```

畫面像數與實際物理距離的比例

- **Box2D 的標準物理單位**
 - 長度 (距離)：公尺 (Meter, m)
 - 質量 (重量)：公斤 (Kilogram, kg)
 - 時間：秒 (Second, s)
 - 力：紐頓 (Newton, $N = \text{kg} \cdot \text{m} / \text{s}^2$)
- 為什麼在 **Love2D** 中數值要設定到 **500** 這麼大？
 - 在預設情況下，**Love2D** 將 **1 個像素 (Pixel)** 直接等同於 **1 公尺 (Meter)**。
 - 這會導致一個反直覺的現象：
 - 我們畫面上寬 **700** 像素的地面，在物理世界裡代表一條 **700 公尺** 長的巨型跑道。
 - 半徑 **25** 像素的小球，代表一顆 **直徑 50 公尺**（相當於 **15 層樓** 高）的超巨大隕石。
 - 因為形狀巨大，透過密度計算出來的小球質量（重量）會非常驚人（可能重達數千公斤）。
 - 要推動一顆幾千公斤的超巨型球體，如果只施加 **1 或 2 紐頓** 的力，它根本紋風不動。這就是為什麼我們必須把力的大小（**speed**）設定到 **500** 甚至更高，小球才會有明顯的移動。
- 如何讓單位變得更符合現實？
 - 如果您希望物理模擬更精準（例如製作真實比例的賽車或彈珠台），可以使用 **love.physics.setMeter** (像素) 縮放比例

遊戲幀率（FPS）與現實時間的轉換

- 如同像數與距離的比例關係
- 現實世界中物體每秒下落 9.81 公尺，但在 800 像素寬的螢幕上，下落 9.81 像素只不過是挪動了幾個像素點。因此，我們必須對這個數值進行「放大比例」：
- 乘以 60 的物理意義，其實是暗示：「我們將 60 像素定義為現實生活中的 1 公尺」。
- 重力加速度 9.81 m/s^2 乘以 60 px/m ，就等於 588.6 px/s^2 。這代表物體下落時，每秒速度會增加 588.6 個像素，這樣在 600 像素高的螢幕裡，下墜感就會非常真實
- 解法：`love.physics.setMeter(60)`：畫面上60個像素代表1公尺